Shenzhen Hi-Link Electronic Co., Ltd.

HLK-LD6004

3D Presence Radar Module
Communication Protocol

Version: V1.0 — September 2025

Shenzhen Hi-Link Electronic Co., Ltd.

Table of Contents

1. Purpose

2. Scope

3. Data Communication Link Conventions

3.1 Communication Link Overview

3.2 Communication Message Examples

4. Protocol Specification

4.1 Protocol Structure Diagram

4.2 Protocol Structure Overview

4.3 TF Frame Parsing Flow

4.4 TF Example Frame Parsing Detail

5. General Message Types

5.1 Query Firmware Status (TYPE: 0xFFFF)

5.2 Return Firmware Status (TYPE: 0xFFFF)

5.3 Enter OTA Upgrade (TYPE: 0x3000)

5.4 OTA Upgrade (TYPE: 0x3000)

6. Presence Detection 2D/3D Project

6.1 Set Control Command 0x0201

6.2 Set Zone Coordinates 0x0202

6.3 Set Hold Delay Time 0x0203

6.4 Set Z-Axis Range 0x0204

6.5 Set Low-Power Sleep Time 0x0205

6.6 Set Stay Zone Target Lifecycle 0x0206

6.7 Set UART Output Interval 0x0207

6.8 Report Personnel Position 0x0A04/0x0A08

6.9 Report Zone Occupancy 0x0A0A

6.10 Report Zone Coordinates 0x0A0B/0x0A0C/0x0A16

6.11–6.22 Additional Report Messages

7. Programming Interface

1. Purpose

This document defines the radar module communication protocol for the HLK-LD6004 3D Presence Radar Module.

2. Scope

This document applies to project developers, maintenance personnel, testers, and customers.

3. Data Communication Link Conventions

3.1 Communication Link Overview

The **server side** refers to the radar module; the **client side** refers to the host/PC application.
Server communication modes: **Active Upload** and **Passive Upload**.
Client communication mode: **Active Downlink**.

3.2 Communication Message Examples

Message Type: Query Firmware Status (TYPE: 0xFFFF)

Communication Mode: Active Downlink

Field	Bytes	Type	Description
SOF	1	uint8	Start-of-frame byte
ID	2	uint16	Frame ID
LEN	2	uint16	Data frame length
TYPE	2	uint16	Frame type
HEAD_CKSUM	1	uint8	Header checksum

Example TF frame: `01 00 01 00 00 FF FF FF`

Message Type: Return Firmware Status (TYPE: 0xFFFF)

Communication Mode: Passive Upload

Field	Bytes	Type	Description
SOF	1	uint8	Start-of-frame byte
ID	2	uint16	Frame ID
LEN	2	uint16	Data frame length
TYPE	2	uint16	Frame type
HEAD_CKSUM	1	uint8	Header checksum
DATA	N	-	See DATA definition below
DATA_CKSUM	1	uint8	Data checksum

DATA field definition:

Bytes	Type	Field	Description
1	uint8	project_name	0=Presence, 1=Respiration, 2=Gesture, 3=Ranging, 4=People Count, 5=3D Point Cloud
1	uint8	major_version	Radar firmware major version

Bytes	Type	Field	Description
1	uint8	sub_version	Radar firmware sub version
1	uint8	modified_version	Radar firmware revision version

Example TF frame: `01 04 AF 00 04 FF FF 51 06 03 0A 00 F0`

Message Type: Upload Target Information (TYPE: 0x0A08)

Communication Mode: Active Upload

Field	Bytes	Type	Description
SOF	1	uint8	Start-of-frame byte
ID	2	uint16	Frame ID
LEN	2	uint16	Data frame length
TYPE	2	uint16	Frame type
HEAD_CKSUM	1	uint8	Header checksum
DATA	N	-	See DATA definition below
DATA_CKSUM	1	uint8	Data checksum

DATA field definition (repeated N times for N targets):

Bytes	Type	Field	Description
4	int32	target_num	Number of detected targets
4	float	x	X coordinate (meters)
4	float	y	Y coordinate (meters)
4	float	z	Z coordinate (meters)
4	int32	dop_idx	Doppler velocity index
4	int32	cluster_id	Cluster target ID
...	...	...	Repeated for each of N targets

Example TF frame: `01 00 06 00 04 0A 08 FE 00 00 00 00 FF`

4. Protocol Specification

4.1 Protocol Structure Diagram

The TF frame consists of the following fields in order: SOF (1B) | ID (2B) | LEN (2B) | TYPE (2B) | HEAD_CKSUM (1B) | DATA (N bytes, little-endian) | DATA_CKSUM (1B). All header fields use big-endian ordering; DATA uses little-endian.

4.2 Protocol Structure Overview

Field	Bytes	Endian	Description
SOF	1	Big	Start of a TF data frame. Fixed value: 0x01
ID	2	Big	Packet sequence number; can be used to detect continuity between client and server frames
LEN	2	Big	Total data frame length. Default max: 1024 bytes
TYPE	2	Big	Message type of the entire data frame; different types represent different functions
HEAD_CKSUM	1	Big	Header checksum: XOR of all bytes from SOF to TYPE, then bitwise NOT (~)
DATA	N	Little	Payload data. Length is determined by LEN field
DATA_CKSUM	1	Big	Data checksum: XOR of all DATA bytes, then bitwise NOT (~)

Note: SOF through HEAD_CKSUM and DATA_CKSUM use big-endian for easy inspection of TYPE and LEN. DATA bytes use little-endian, matching processor storage order for easy type-casting.

4.3 TF Frame Parsing Flow

The parser is a state machine that advances through states: SOF → ID → LEN → TYPE → HEAD_CKSUM → DATA → DATA_CKSUM → SOF. Each incoming byte is XOR'd into the running checksum accumulator. On reaching HEAD_CKSUM state, the accumulated XOR result is bitwise-inverted and compared against the received byte. The same logic applies for DATA_CKSUM at the end of the payload.

4.4 TF Example Frame Parsing Detail

Example frame bytes: `01 00 00 00 01 30 00 CF FF 00`

Field	Value	Notes
SOF	0x01	Start byte
ID	0x0000	Frame sequence 0
LEN	0x0001	Data length = 1 byte
TYPE	0x3000	OTA upgrade type
HEAD_CKSUM	0xCF	~(0x01^0x00^0x00^0x00^0x01^0x30^0x00) = 0xCF
DATA	0xFF	1 byte payload (OTA pending)
DATA_CKSUM	0x00	~(0xFF) = 0x00

Float extraction example: If x_point bytes are 0x66 0x66 0xA2 0x41 (little-endian), assemble as uint32 = 0x41A26666, then reinterpret as float → 20.3 m.

```
unsigned int param = 0x41A26666;
float res = *(float*)&param; // res = 20.3
```

5. General Message Types

5.1 Query Firmware Status (TYPE: 0xFFFF)

Communication Mode: Active Downlink (Host → Radar)

Field	Bytes	Type	Description
SOF	1	uint8	Start byte
ID	2	uint16	Frame ID
LEN	2	uint16	Data length (0x0000 — no payload)
TYPE	2	uint16	0xFFFF
HEAD_CKSUM	1	uint8	Header checksum

Example: `01 00 01 00 00 FF FF FF`

5.2 Return Firmware Status (TYPE: 0xFFFF)

Communication Mode: Passive Upload (Radar → Host)

Bytes	Type	Field	Description
1	uint8	project_name	0=Presence, 1=Respiration, 2=Gesture, 3=Ranging, 4=People Count, 5=3D Point Cloud
1	uint8	major_version	Major version
1	uint8	sub_version	Sub version
1	uint8	modified_version	Revision version

Example: `01 04 AF 00 04 FF FF 51 06 03 0A 00 F0`

5.3 Enter OTA Upgrade (TYPE: 0x3000)

Communication Mode: Active Downlink. No DATA payload.

Example: `01 00 01 00 00 30 00 CF`

5.4 OTA Upgrade Response (TYPE: 0x3000)

Communication Mode: Passive Upload. DATA = 1 byte [tmpCodeInfo]: 0xFF = awaiting upgrade.

Example: `01 00 03 00 01 30 00 CC FF 00`

6. Presence Detection 2D/3D Project

Important Notes:

1. After sending any TF frame, the radar will first reply with the same TYPE but no DATA, confirming receipt. If no reply, resend the configuration message.

2. When low-power mode (no-person) is enabled, the radar enters sleep. To send config, first pull UART0 RX0 pin low to wake it. Confirm success by reading back; resend if no response.

3. After the host sends data or the radar is woken, it operates in normal mode for 10 seconds before re-entering low-power mode if no person is detected.

6.1 Set Control Command (TYPE: 0x0201)

Communication Mode: Active Downlink only. DATA = 4-byte int32 [command].

Example: `01 00 00 00 04 02 01 F3 01 00 00 00 FE`

Command	Function
0x01	Auto-generate interference zones
0x02	Get interference, detection, and stay zone coordinates
0x03	Clear interference zones
0x04	Reset detection zones
0x05	Get hold delay time
0x06	Enable point cloud display
0x07	Disable point cloud display
0x08	Enable target display
0x09	Disable target display
0x0A	Set detection sensitivity: Low
0x0B	Set detection sensitivity: Medium
0x0C	Set detection sensitivity: High
0x0D	Get detection sensitivity status
0x0E	Set trigger speed: Slow (5s)
0x0F	Set trigger speed: Medium (1s)
0x10	Set trigger speed: Fast (0.5s)
0x11	Get trigger speed status
0x12	Get Z-axis range (3D only)
0x13	Set installation: Ceiling mount (3D only)
0x14	Set installation: Wall mount (3D only)
0x15	Get installation mode
0x16	Set mode: Low-power when no person detected
0x17	Set mode: Normal detection mode
0x18	Get current working mode
0x19	Get low-power sleep time
0x1A	Reset no-person state

Command	Function
0x1B	Radar off: P20/OUT outputs HIGH
0x1C	Radar off: P20/OUT outputs LOW
0x1D	P20 output: HIGH when person, LOW when no person (default)
0x1E	P20 output: LOW when person, HIGH when no person
0x1F	Get P20 output state
0x20	P20 no-action: continuous LOW
0x21	P20 no-action: continuous HIGH
0x22	P20 pulse: 500ms LOW on state change
0x23	P20 pulse: 500ms HIGH on state change
0x24	Set mode: Strong reflection scene mode
0x25	Clear stay zones
0x26	Get stay zone target lifecycle
0x27	Get UART output interval

6.2 Set Zone Coordinates (TYPE: 0x0202)

Sets coordinates for interference zones (ID 0–3), detection zones (ID 4–7), and stay zones (ID 8–11). Only one zone can be configured per message. For 2D use, set z_min = -6m and z_max = 6m.

Field	Bytes	Type	Description
area_id	4	int32	Zone ID: 0-3=interference, 4-7=detection, 8-11=stay
x_min	4	float	Minimum X coordinate (meters)
x_max	4	float	Maximum X coordinate (meters)
y_min	4	float	Minimum Y coordinate (meters)
y_max	4	float	Maximum Y coordinate (meters)
z_min	4	float	Minimum Z coordinate (meters)
z_max	4	float	Maximum Z coordinate (meters)

6.3 Set Hold Delay Time (TYPE: 0x0203)

DATA = 4-byte uint32 [pwm_delay]: delay time in seconds before switching to 'no person' state. **Default: 30 seconds.**

6.4 Set Z-Axis Range (TYPE: 0x0204) [3D only]

DATA = float z_min + float z_max (8 bytes total). Sets the overall Z detection range.

6.5 Set Low-Power Mode Sleep Time (TYPE: 0x0205)

DATA = 4-byte uint32 [waitingPeriod]: sleep interval in milliseconds when no person is detected. **Default: 500ms.**

6.6 Set Stay Zone Target Lifecycle (TYPE: 0x0206)

DATA = 4-byte uint32 [stayLife]: lifecycle value. Decrements by 20 per second. **Default: 300 (equivalent to 15 seconds).**

6.7 Set UART Output Interval (TYPE: 0x0207)

DATA = 4-byte uint32 [outputInterval]: output every N frames. **Default: 1 (every frame). Example: setting 10 = output every 10 frames.**

6.8 Report Personnel Position (TYPE: 0x0A04 / 0x0A08)

Radar → Host. 0x0A04 = tracked target data; 0x0A08 = point cloud data. In 2D mode, Z coordinate is always 0.

Field	Bytes	Type	Description
target_num	4	int32	Number of detected targets (N)
x	4	float	X coordinate in meters (repeated N times)
y	4	float	Y coordinate in meters (repeated N times)
z	4	float	Z coordinate in meters (0 in 2D mode, repeated N times)
dop_idx	4	int32	Doppler velocity index (repeated N times)
cluster_id	4	int32	Cluster target ID (repeated N times)

6.9 Report Zone Occupancy (TYPE: 0x0A0A)

Reports whether a person is present in each of the 4 detection zones. 1 = occupied, 0 = empty.

Field	Bytes	Type	Description
detection_state_area0	4	uint32	Zone 0: 1=person present, 0=no person
detection_state_area1	4	uint32	Zone 1: 1=person present, 0=no person
detection_state_area2	4	uint32	Zone 2: 1=person present, 0=no person
detection_state_area3	4	uint32	Zone 3: 1=person present, 0=no person

Example: `01 00 00 00 10 0A 0A F3 01 01 00 01 FE`

6.10 Report Zone Coordinates (TYPE: 0x0A0B / 0x0A0C / 0x0A16)

Triggered by command 0x0201 with data=0x02. Returns three messages: 0x0A0B = 4 interference zone coordinates; 0x0A0C = 4 detection zone coordinates; 0x0A16 = 4 stay zone coordinates. Each zone contains: x_min, x_max, y_min, y_max, z_min, z_max (all floats, meters).

6.11–6.22 Additional Report and Configuration Messages

TYPE	Name	Direction	Key Data
0x0A0D	Report Hold Delay Time	Radar→Host	uint32 pwmDelayTimer (seconds)
0x0A0E	Report Detection Sensitivity	Radar→Host	uint8: 0=Low, 1=Medium, 2=High
0x0A0F	Report Trigger Speed	Radar→Host	uint8: 0=Slow, 1=Medium, 2=Fast
0x0A10	Report Z-Axis Range (3D)	Radar→Host	float z_min, float z_max (meters)
0x0A11	Report Installation Mode	Radar→Host	uint8: 0=Ceiling, 1=Wall
0x0A12	Report Working Mode	Radar→Host	0=Normal, 1=Low-power, 2=Radar off HIGH, 3=Radar off LOW, 4=Stro
0x0A13	Report Low-Power Sleep Time	Radar→Host	uint32 waitingPeriod (ms)
0x0A14	Report Low-Power Mode State	Radar→Host	uint8: 0=In low-power state, 1=Normal mode
0x0A15	Report GPIO (P20) State	Radar→Host	0=Hi/Lo, 1=Lo/Hi, 2=Cont.LOW, 3=Cont.HIGH, 4=Pulse LOW 500ms, 5
0x0A17	Report Stay Zone Lifecycle	Radar→Host	uint32 stayLife
0x0A18	Report UART Output Interval	Radar→Host	uint32 outputInterval
0x0F0F	Set Bluetooth/UART Baud Rate	Host→Radar	uint8 baud_index: 0=9600, 1=115200(default), 2=128300, 3=256000, 4

7. Programming Interface

7.1 Encoding a TF Message

`void tinyFramefTx(TF_TYPE type, uint8 *data, TF_LEN len);`

Parameters: **type** (uint16) — message type (e.g. 0x0A10); **data** — pointer to payload bytes; **len** (uint16) — payload length.

7.2 Decoding a TF Message

`TinyFrameRx tinyFramefRx(void);`

On successful reception, returns a TinyFrameRx struct containing the parsed frame fields.

A. Extracting float from little-endian DATA bytes:

Assemble the 4 bytes into a uint32 (little-endian), then reinterpret-cast to float:

```
int main(void) {
    unsigned int param = 0x41A26666;  // bytes: 0x66 0x66 0xA2 0x41
    float res = *(float *)&param;
    printf("data: %f\n", res);  // prints: 20.300000
    return 0;
}
```

B. Checksum calculation (HEAD_CKSUM and DATA_CKSUM):

HEAD_CKSUM covers bytes from SOF up to (not including) HEAD_CKSUM.
DATA_CKSUM covers all DATA bytes.
Algorithm: XOR all bytes in the range, then bitwise NOT (~).

```
unsigned char getCksum(unsigned char *data, unsigned char len) {
    unsigned char ret = 0;
    for (int i = 0; i < len; i++) {
        ret = ret ^ data[i];
    }
    ret = ~ret;
    return ret;
}
```

7.3 Sample Code

Demo code for parsing TF frame data is available for Linux (C), Keil µVision5 (C), and Python environments. Contact Hi-Link sales to obtain the demo package.

© 2025 Shenzhen Hi-Link Electronic Co., Ltd. — Document Version V1.0 — Released 2025-09-25